

Next Steps: Feasibility and Plan

Your four ideas evaluated, plus my additions, a proposed roadmap and first sketches of the editor app

Project C:\Games\FIFA-14-Local-FUT-main **Date** 25 Sep 2026 **Status** discussion only, nothing implemented

Verdicts at a glance

Idea	Verdict	Size
1. Created players in FUT	Showing them as cards is realistic: the project already has a client-database injector for the 42 Legends, and your saves contain readable EA databases. Two unknowns need tests first: the game rejected the last attempt to <i>grow</i> the player table, and it's unknown whether new players work in matches .	research first L
2. Custom glare	Per-type glare color, on/off and shape (rare or legend mask): easy. It's the same kind of edit as text colors, because the glare color is just an RGB value picked in one function. Replacing the glare artwork for <i>all</i> cards is a texture swap inside card.big. Different artwork <i>per type</i> is hard (new movie resources).	easy / hard for per-type art S to M
3. More than 4 colored types	Yes. The limit is simply bytes in the footer-color function. Quick fix: 6 types. Proper fix: one shared "style table" function that serves text colors and glare for every type, which does idea 2 in the same step.	6: easy all: medium S / M
4. GUI editor (Python + Qt Designer)	Yes, and recommended as the hub for everything else. All the data it needs is reachable from Python (SQLite, JSON, DDS, the generator as a library). Card preview works without the game. Name and photo editing depend on the client DB and head archives (see idea 1).	medium L (in stages)

Recommended order

1. Quick wins in the generator: allow all 36 type slots, per-type glare (color, off, shape), 6 colored types. **2.** Editor app v1: browse and edit club players, card types, colors and glare with live preview. **3.** Shared style table, removing the color limit. **4.** Research track for custom players, starting with three small in-game tests. Details and a timeline are in section 6.

What this is based on

Everything below was checked against the actual files for this document, not guessed:

- the glare color function was disassembled;
- all 54 textures inside card.big were decoded to find the glare artwork;
- the 4-color limit was measured by running the generator with 0 to 8 colored types;
- your save files were checked for EA databases;
- the Legends injector and the launcher were read;
- the player-photo archives and the server database fields were listed.

Where something is still unknown, it's marked as a test to run.

1. Created players in FUT

Goal: a player you create in FIFA's menu (or define yourself) appears as a FUT card you own, can put in your squad, and ideally can play with.

What a FUT card is made of



Figure 1. Four ingredients. Green ones are within reach; amber and red need research.

Evidence for the split: the payload's `name` is ignored. Your slot-6 card says "Romelu Lukaku" in the database but shows Nainggolan, because the game reads names from its own database by asset id. Ratings and stats, on the other hand, very likely come from the server item. Benteke's item carries a 95 rating with custom stats, and FUT special cards always had ratings different from the base game, so the client must take them from the item. Test 3 below confirms it.

What already exists

- `tools\patch_fifa14_fut_legends_db.py` (1,800 lines) reads and writes FIFA's database format (following the public DBM Studio layout). It can add name strings with the DB's own Huffman encoding, clone a "donor" row, and even grow the players table.
- `server\fifa14-legend-catalog.v24013.json` shows the server-side format for extra players: asset id, rating, position, team, league, nation, rareFlag, attributes.
- But:** the launcher now *undoes* this injection on every start. Its comment says an earlier version produced a `cards_ng_db` "that our parser verified but the retail game rejected during startup". The table-growing step is the prime suspect.
- Your saves contain uncompressed EA databases: the career save holds a 3.6 MB database with 29 tables, the settings saves small single-table ones. So **reading a created player out of a save is realistic** with the existing parser. What isn't known yet is *which* file FIFA 14 writes created players to (a squad save, the career save, or elsewhere).

Options

A. Recycle existing rows (recommended first)

medium · lowest risk

Pick players you'll never use (the table has thousands) and overwrite their name and identity with your custom player's. The table doesn't grow, so the format stays exactly as the game expects. Cost: the recycled player is gone from the game. For matches it depends on the unknown above: best case the new player looks generic, worst case they look like the old player.

B. Find out why the grown table was rejected

research

Candidates: a checksum in the DB header or per table, a fixed-size buffer the game loads the DB into, or the archive record's size. Diffing a grown DB against the original (header fields, table offsets, CRCs) should show which. If it's a fixed buffer, only option A works.

C. Import created players from saves

medium, after A or B

Create a test player in FIFA, save, then diff the Documents folder to see which file changed. Read that player's row (name, ratings, positions, appearance) and feed it into A or B plus a server item. The GUI (idea 4) gets an "Import from save" button.

D. Players defined entirely in the editor

medium, same plumbing

The same pipeline without FIFA's creator: type name, stats and position in the GUI, pick a photo, and it goes through A or B. Useful for icons and legends-style cards too.

First three tests (cheap, answer the big unknowns)

1. **Where do created players go?** Create one player, save, and list which files in `Documents\FIFA 14` changed.
2. **Row recycling:** rename one obscure player in `cards_ng_db` (same-size change, no table growth) and check the card shows the new name. Then play one FUT match with that card to see what the match does.
3. **Server-side fields:** change one owned player's rating, one stat, club and nation in the payload and confirm all four show on the card. The GUI's field list depends on this.

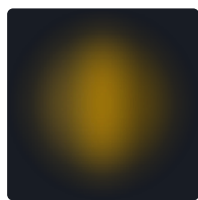
2. Custom glare

How the glare actually works

The shine is a separate layer on the card (`mcRareShine`) built from two textures that live inside `card.big`, cut to the card's shape by a mask:



Glint (entry 57, 128×32): the white streak that sweeps across.



Flare (entry 61, 256×256): the soft glow.



Masks (entries 52 and 54 for big cards, 64 and 66 collapsed): the card outline. The code picks a mask frame called `'rare'` or `'legend'`.

The **color** is not in the textures. `getShineColor()` returns one RGB value per card type, and the game tints the glint and flare with it at 60% opacity. The retail values:

Type	Tint	Type	Tint	Type	Tint
Rare gold	 F0C708	Purple (4)	 C6AAFF	MOTM orange (8)	 FFD5B4
Rare silver	 E7E7E5	TOTY blue (5), RB (6), TOTS (11)	 8DBFFF	FUTTIES pink (9)	 FFA5C8
Rare bronze	 E3A77D	Green (7)	 56D150	Teal (10)	 7AE8E9
TOTW (3)	 000000	Legend (12)	 DBC274	anything else	 000000

What can be customized

Customization	How	Difficulty
Glare on or off per type	The shine switch from this week's fix, driven by config instead of "never for new types".	easy
Glare color per type ("gold", "silver", any RGB)	Add your types to <code>getShineColor</code> and its three siblings (fake big, small, fake small). Same technique as text colors. Presets can be the retail values above.	easy
Mask shape per type (<code>rare</code> or <code>legend</code>)	Choose which existing mask frame to show.	easy

Intensity, speed, frequency	Opacity (60%) and the timing constants are single numbers. Changing them for all cards is a tiny edit; per type needs a little code.	global easy per type medium
Your own glare artwork, all cards	Replace the glint and/or flare texture inside card.big with a same-size DXT5 image. The patch tool already handles byte edits of that size.	easy-medium
Different artwork per type	Needs new textures <i>and</i> new movie objects to show them. That's Apt movie editing beyond byte patching (see 5.4).	hard

Proposed config (one entry per type; a missing entry means no glare, as today):

```
"glare": {
  "17": {"color": "#F0C708", "mask": "rare"},      // classic gold rare shine
  "19": {"color": "#C6AAFF"},                    // purple tint, default mask
  "20": {"preset": "silver"}                      // named presets = retail values
}
```

3. More than 4 colored types

Why the limit is 4

Each colored type adds a small block of code to three functions, and each function has to stay exactly its original size (that's what makes the edits safe). Measured with the real generator:

Function	Room	Used with 0 custom types	Cost per colored type	Max types
Card::SetMainFontColors	668 B	499 B	28 B	6
Card::setFooterFontColor	472 B	367 B	24 B	4 (the bottleneck)
CardSmall color function	842 B	614 B	36 B	6

Options

A. Footer follows text unless set

easy · limit becomes 6

Most types won't want a separate stats color. If the footer function only handles types with an explicit `footer` and otherwise uses the text color already chosen, it stops being the bottleneck. The limit rises to 6 (main and small).

B. One shared "style table" function (recommended)

medium · all types

EA's `getShineColor()` is very verbose: 831 bytes in card.big and 783 in cardsmall, for what is really a small lookup table. Rewritten compactly, it becomes a helper that answers "which color does type *N* use for header, text, footer or glare?" from a table. The three color functions just call it (about 10 bytes per call), so the per-type cost moves into the helper, where a type costs roughly 8 bytes (small indexes into a shared color list). That covers **every new type**, and it's the same function idea 2 needs for glare colors, so both ideas become one project. It's a proper function call, so no tricks with jumps between functions.

C. Grow the movies (platform research)

research · unlimited

Everything so far rewrites code in place. If card.big could be *extended* with new code, new strings and new images, every limit disappears, including per-type glare art. That means rebuilding the Apt movie container. The format is shared with other EA games of the era (C&C Generals, C&C 3), which have open-source readers (for example the OpenSage project), so there's prior work to lean on. FIFA's version (1:7:4) may differ, so this is a research track, not a plan yet.

Related quick win: 36 type slots instead of 16

The background switches compare the type against each new type one by one (12 bytes each), which caps them at 16. But every new type does the same thing (*id* = *type*, or *type* + 100 on small cards). One range check (the same helper the glare fix uses) handles a whole run of types in about 24 bytes. That makes all 36 slots from 14 to 49 usable. It's a small change to the generator.

4. GUI editor app

Goal: one desktop app to browse your club, edit players, manage card types (art, text colors, glare) and preview cards, without touching JSON or hex. You know Python and Qt Designer, so the plan uses **PySide6** (official Qt for Python, LGPL) with **.ui files from Qt Designer**. PyQt6 would work the same.

Architecture

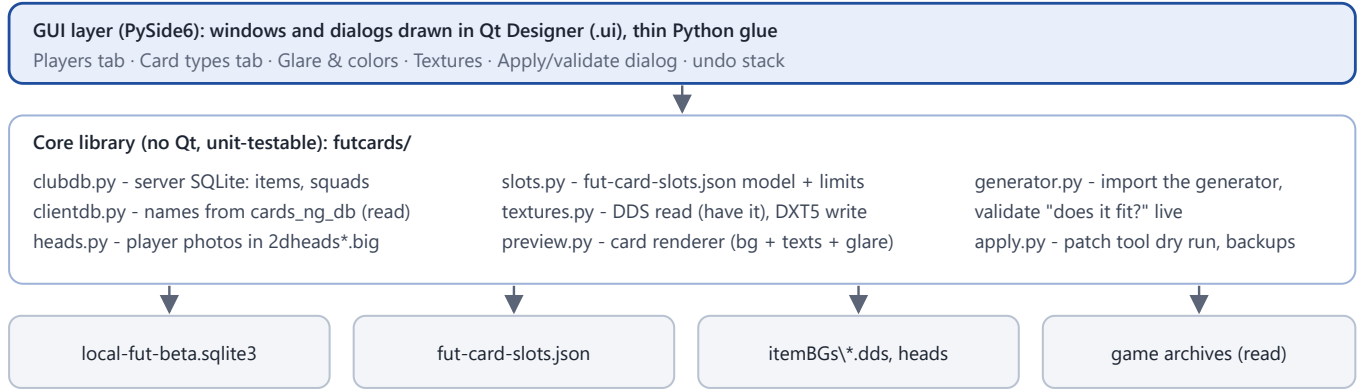


Figure 2. Keep all logic in a plain Python library so it can be tested and reused by command-line tools; the Qt layer only displays and edits.

What the app can edit, and how sure we are

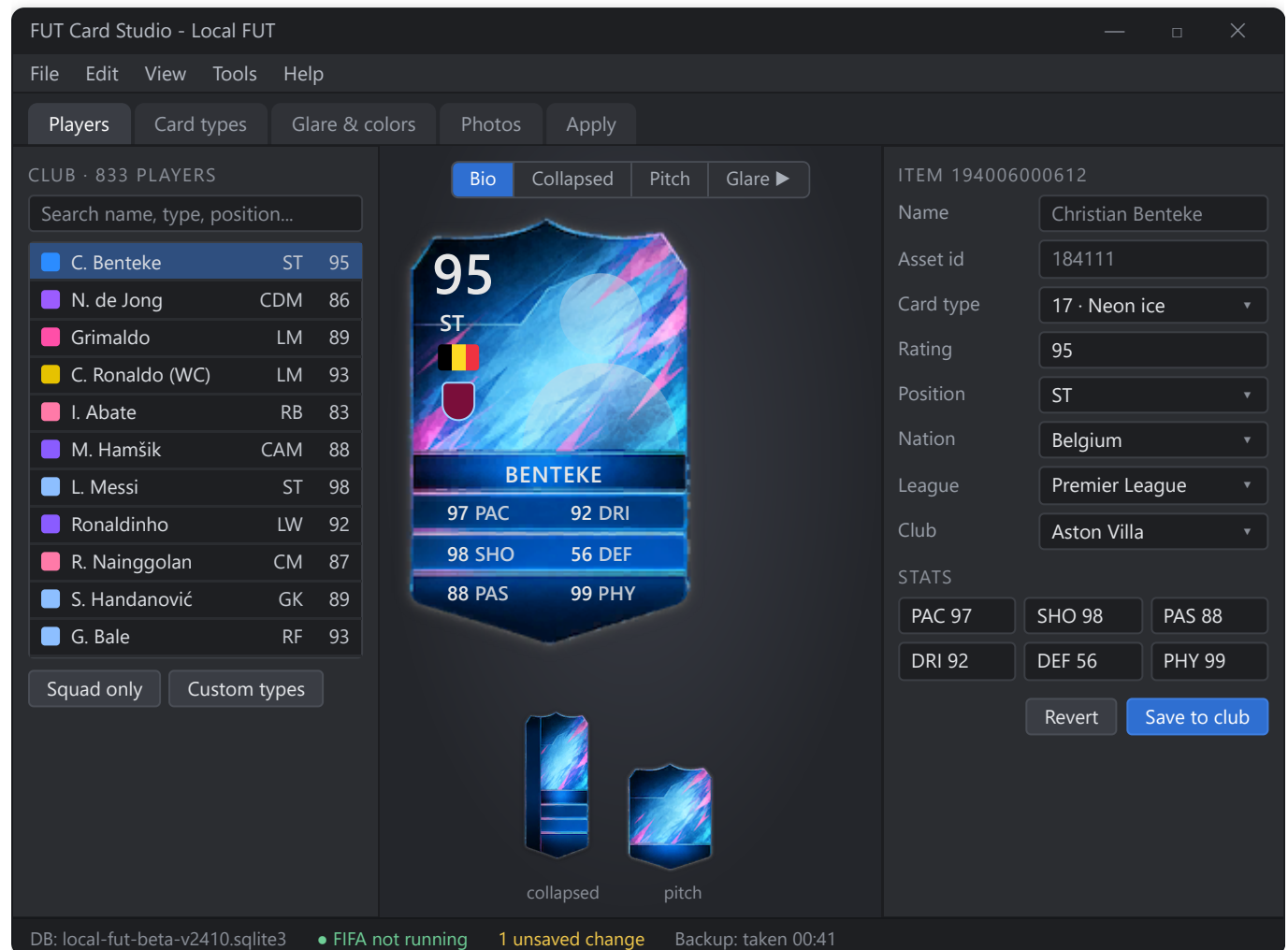
Field	Stored in	Status
Card type (rareFlag)	payload rareflag + rareFlag	proven
Rating, 6 stats	payload rating , attributeArray	very likely; confirm with test 3
Position, club, league, nation	payload preferredPosition , teamid , leagueId , nation	likely (FUT items carry them); test 3
Name	client DB (cards_ng_db)	idea 1 territory; read-only in v1
Photo	p<id>.dds in 2dheads*.big / headswc	feasible (same archive method); v2
Card-type art, text colors, glare	itemBGs + fut-card-slots.json	proven (glare after idea 2)

Rules the app should enforce: back up the SQLite database before the first write of a session; show the generator's verdict (fits or not) before saving the config; only run the patch tool's --apply when FIFA is closed; resolve player names from the client DB, never from the payload's stale name .

4.1 Sketches of the app

Static mockups with your real textures and club data. The card overlay positions are approximate; the real renderer would be tuned against in-game screenshots.

Players tab



Names in the list come from the client DB, never from the payload (which still says "Lukaku" for Nainggolan).

Card types tab

FUT Card Studio - Local FUT

PlayersCard typesGlare & colorsPhotosApply

NEW TYPES

14 · Holo ✓ A

15 · Violet ✓ A

16 · Pink hex ✓ A

17 · Neon ice ✓

18 · Shard ✓ A

19 · Cosmic ✓

20 · (empty) -

21 · (empty) -

22 ... 49

+ New type

✓ = art present, A = custom text colors

TYPE 17 · TEXTURES (ITEMBGs)



large ✓



1017 ✓



117 ✓



317 ✓

Import PNG...

Make collapsed + small from large

Open folder

Imports are converted to DXT5 without mipmaps and checked for size (65,664 / 32,896 / 16,512 bytes).

PLAYERS USING TYPE 17

C. Benteke (ST 95)

NAME (EDITOR ONLY)

Neon ice

TEXT COLORS

Header

☐ default

Text

☐ default

Footer

= text

GLARE

Mode

None

Pre-cache

☐ with EA's set

CAPACITY

Types

15 / 36

Colored types

4 / 4

Pre-cached

3 / 4

fut-card-slots.json

generator: all 12 blocks fit

Changes apply on next launch

Type names such as "Neon ice" would be stored only in the editor's own settings (the game has no names for these types). Capacity meters come from the generator, so the app can't save something that won't fit.

Glare & colors tab

FUT Card Studio - Local FUT

PlayersCard typesGlare & colorsPhotosApply



89
LM


GRIMALDO
92 PAC 90 DRI
88 SHO 77 DEF
89 PAS 70 PHY

Preview plays the sweep with the chosen tint, mask and speed. It uses the real glint and flare textures read from card.big.

TYPE 16 · PINK HEX

TEXT COLORS

Header

 #FF10F0

Text

☐ #E6E6E6

Footer

= text

GLARE

Enabled

☒ on

PRESETS

Gold

Silver

Bronze

Legend

Purple

Green

Pink

Teal

Custom...

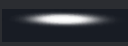
Mask

rare

Intensity

60 % (all cards)

GLOBAL ARTWORK



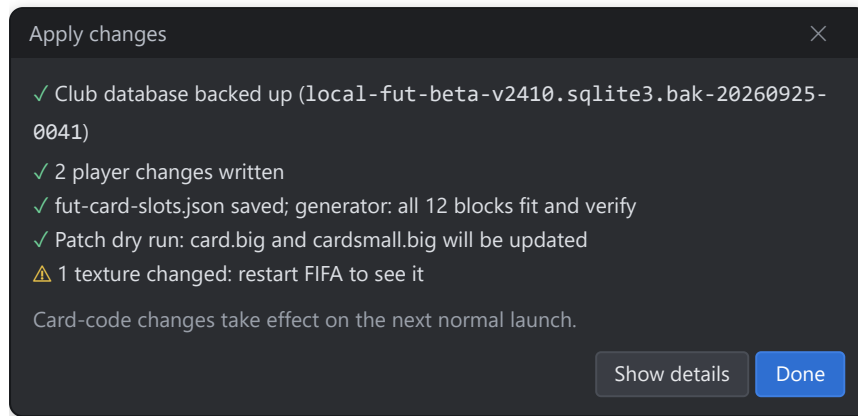
Replace...

Changes the glint/flare for every shining card.

Preview only - nothing written yet

fits

Apply dialog



5. My additional ideas

Idea	What and why	Effort
5.1 36 type slots	Range checks in the background switches (section 3). Removes the 16-type cap; everything from 14 to 49 works.	S
5.2 One-master texture pipeline	Design only the 256×256 card; the app derives the collapsed (128×256) and pitch (128×128) versions, with manual override. PNG in, DXT5 out, via Microsoft's <code>texconv</code> (DirectXTex) or ImageMagick. Reading DDS already works in pure Python (written for this doc).	M
5.3 Card gallery / share images	The offline renderer from the app can export PNGs of cards (for Discord or for comparing designs), and a gallery page of all your types.	S once the renderer exists
5.4 Reusable Apt toolkit	Turn the generator's decoder, assembler and checks into a small package (disassemble any function, patch any region). Every future UI tweak (other screens, not just cards) gets cheaper. It's also the base for the movie-growing research (3C).	M
5.5 Custom types in packs	The server decides pack contents (<code>pack-catalog</code> , <code>pack-weights</code> JSON). A "promo" pack, or a small chance of a custom-type card in normal packs, makes your designs show up in play, not just by editing.	M
5.6 Promo rotations	A server-side job that, say weekly, turns chosen players into a promo type (your own TOTW-style event) and later reverts them. Pure database work.	M
5.7 Photo manager	Swap or add player photos (<code>p<id>.dds</code>) the same append-and-repoint way as <code>card.big</code> . You already did one by hand (Ronaldinho's World Cup head); this makes it a button.	M
5.8 Design profiles	Save and switch whole sets of card-type art + colors + glare ("season 1", "neon pack"), since everything is files plus one JSON.	S
5.9 Launch health check	After each launch, confirm the edits were applied and nothing failed (the launcher output already has the facts). Show it in the app's status bar.	S
5.10 Type label text	Research whether the item detail screen shows a type label ("Rare", "TOTW") that could say your type's name. Unknown; a nice-to-have.	research

Sizes: **S** = about one working session like ours, **M** = two or three, **L** = several, spread over milestones. These are rough guides, not promises. Research items can end in "not possible".

6. Proposed roadmap

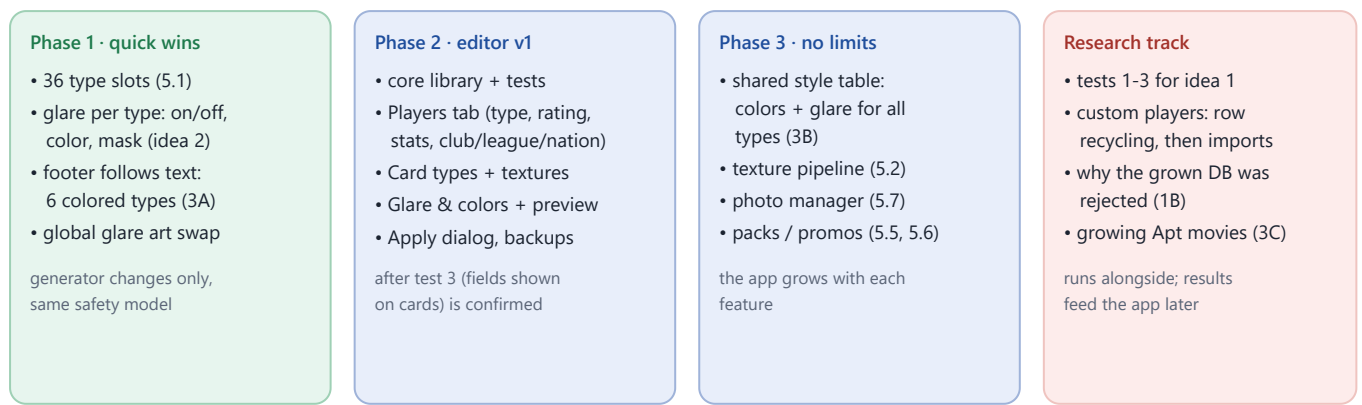


Figure 3. Phases 1 to 3 build on proven techniques; the research track answers the open questions in parallel.

Open questions, in the order I'd answer them

#	Question	How to answer
1	Do payload rating, stats, club, league and nation show on the card?	Edit one owned player, open the card (test 3).
2	Where does FIFA 14 save created players?	Create one, diff Documents\FIFA 14 (test 1).
3	Does a renamed row in cards_ng_db load, and what happens in a match?	Rename one obscure player; play one match (test 2).
4	Why did the grown player table fail?	Diff grown vs original DB headers, CRCs, offsets.
5	Can Apt movies be extended safely?	Study OpenSage-style Apt readers; one harmless in-game test.

Suggested next session

Phase 1 in the generator (36 slots, glare per type, 6 colored types), plus test 3, which is a one-minute check on your side. That gets visible results immediately and settles the editor's field list before any GUI code is written.